SBC Summer School 2026

Técnicas em Programação Dinâmica

Bernardo Amorim

31 de janeiro de 2026

Slide

Plano de aula

- DP
- Otimização de espaço
- DP de dígitos
- DP de componentes conexos
- Temos que visitar tudo?
- Lidando com ciclos

Problema Frog 2

Temos $1 \leq N \leq 10^5$ pedras numeradas de 1 a N e a i-ésima pedra tem um valor associado $1 \leq h_i \leq 10^4$. Um sapo começa na posição 1 e deve chegar até a posição N com o menor custo possível. Um sapo na posição i pode ir para as posições $j = i + 1$, $i + 2$, ..., $i + k$, $1 \leq K \leq 100$ com um custo $|h_i - h_j|$, qual é o custo mínimo para o sapo chegar ao seu destino?

- Exemplo: $N = 5$, $K = 3$:

h:	1	3	4	5	2

Problema Frog 2

- Exemplo: $N = 5$, $K = 3$:

$$\mathtt{h} \;\Big|\; 1 \quad 3 \quad 4 \quad 5 \quad 2$$

$$\text{Custo} = |1-3|+|3-4|+|4-5|+|5-2| = 7$$

Problema Frog 2

- Exemplo: $N = 5$, $K = 3$:

$$\mathtt{h} \;\Big|\; 1 \quad 3 \quad 4 \quad 5 \quad 2$$

$$\text{Custo} = |1 - 4| + |4 - 2| = 5$$

Problema Frog 2

- Exemplo: $N = 5$, $K = 3$:

$$\mathtt{h} \;\Big|\; 1 \quad 3 \quad 4 \quad 5 \quad 2$$

$$\text{Custo} = |1 - 3| + |3 - 2| = 3 \text{ (melhor resposta)}$$

Relação de Recorrência

- Ao invés de atacar o problema diretamente, podemos pensar em resolver **subproblemas** e unir suas soluções para resolver o problema maior.
- Caso soubéssemos o custo $c(i)$ para outros índices $i < 4$, qual é o custo de se chegar em 4?
- Exemplo: $N = 5$, $K = 3$:

h	1	3	4	5	2
c	0	2	3	4	?

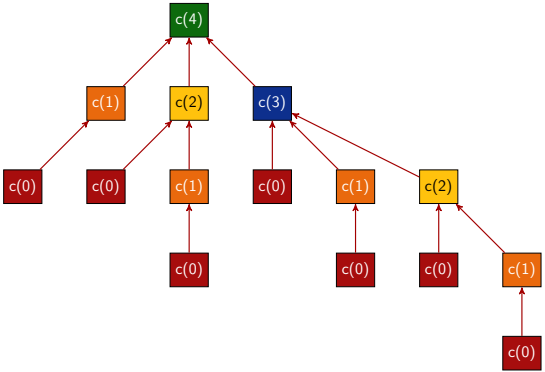
$$c(4) = \min_{1 \leq i \leq 3} c(i) + |h(i) - h(4)|$$

Relação de Recorrência

- Agora que a cara da nossa solução ficou mais clara, podemos escrever uma **relação de recorrência** para as soluções.

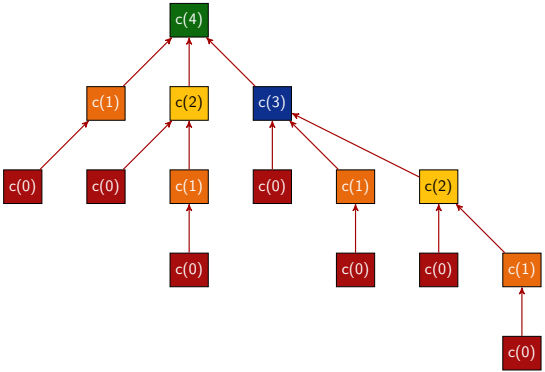
$$c(i) = \begin{cases} 0 & i = 0 \\ \min\limits_{\substack{j \geq 0 \\ i-K \leq j \leq i-1}} c(j) + |h(j) - h(i)| & i > 0 \end{cases}$$

Subproblemas

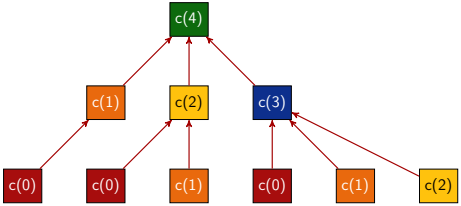


Sem memoização.

Subproblemas



Sem memoização.



Com memoização.

Otimização de espaço

- Via de regra, ao fazer um problema de programação dinâmica mantemos todos os estados.
- Em alguns problemas, pode ser proibitivo guardar todos os estados na memória
- A depender da ordem da DP, não precisamos ter todos os estados o tempo todo.
- Podemos fazer a DP em ordem e sobrescrever (e/ou deletar antigos) os estados à medida que forem necessários

Problema Coin Combinations II

Considere um sistema de moedas consistindo de N moedas ($1 \leq n \leq 100$), cada uma com um valor positivo $c_i (1 \leq c_i \leq 10^6)$. Sua tarefa é calcular o número de formas distintas *ordenadas* tal que você pode produzir a soma $X (1 \leq X \leq 10^6)$ usando as moedas disponíveis.

Por exemplo, se as moedas forem $\{2, 3, 5\}$ e a soma desejada for 9 há 3 formas:

- $2 + 2 + 5$
- $3 + 3 + 3$
- $2 + 2 + 2 + 3$

Problema Coin Combinations II

Os estados úteis na DP são naturais: qual é a soma atual e qual é a moeda que estamos olhando atualmente (faz sentido montarmos as sequências na ordem).

Problema Coin Combinations II

Os estados úteis na DP são naturais: qual é a soma atual e qual é a moeda que estamos olhando atualmente (faz sentido montarmos as sequências na ordem).
Seja $f(i, x)$ o número de formas de gerarmos a soma x usando as moedas $1, \ldots, i$. A resposta, com essa formulação, seria $f(N, X)$ (one based).

$$f(i, x) = \begin{cases} 1 & x = 0 \\ 0 & i = 0, x \neq 0 \\ f(i-1, x) + f(i, x - c_i) & i > 0 \end{cases}$$

Complexidade de tempo e espaço: $\mathcal{O}(C * N)$. Que para o nosso problema $\approx 10^8$, memória demais!

Problema Coin Combinations II

Sempre olhamos o estado i ou $i - 1$.

$$f(i, x) = \begin{cases} 1 & x = 0 \\ 0 & i = 0, x \neq 0 \\ f(\mathbf{i} - \mathbf{1}, x) + f(\mathbf{i}, x - c_i) & i > 0 \end{cases}$$

Se calcularmos os estado da DP em ordem crescente de i, conseguimos reduzir o tamanho da tabela para apenas C ao invés de $X * C$, resolvendo o nosso problema.

Problema Coin Combinations II

Antes da otimização.

```
const int N_MAX = 101, C_MAX = 1e6 + 10, MOD = 1e9 + 7;
int memo[N_MAX][C_MAX]; // f(i, x)

memo[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int x = 0; x < C_MAX; x++) {
        memo[i][x] = memo[i - 1][x]; // não pego
        if (x >= c[i - 1])
            memo[i][x] += memo[i][x - c[i - 1]]; // pego a moeda
        memo[i][x] %= MOD;
    }
}
cout << memo[n][X] << endl;
```

Problema Coin Combinations II

Depois da otimização.

```
int memo[2][C_MAX]; // f(i, x)

memo[0][0] = 1;
for (int i = 1; i <= n; i++) {
    for (int x = 0; x < C_MAX; x++) {
        memo[i%2][x] = memo[(i - 1)%2][x]; // não pego
        if (x >= c[i - 1])
            memo[i%2][x] += memo[i%2][x - c[i - 1]]; // pego a moeda
        memo[i%2][x] %= MOD;
    }
}
cout << memo[n % 2][X] << endl;
```

Nota

Podemos usar $(\mod 2)$ *para alternar entre duas tabelas - a tabela referente à* i *e a referente a* $i - 1$.

Problema Coin Combinations II

Depois da otimização cache friendly.

```
1     int memo[C_MAX]; // f(i, x)
2
3     memo[0] = 1;
4     for (int i = 1; i <= n; i++) {
5         for (int x = 0; x < C_MAX; x++) {
6             if (x >= c[i - 1])
7                 memo[x] += memo[x - c[i - 1]]; // pego a moeda
8             memo[x] %= MOD;
9         }
10    }
11    cout << memo[X] << endl;
```

Nota

Como iteramos em ordem tanto em i quanto em x, podemos sobrescrever a mesma tabela.

Problema Odometer

Quantos números entre X e Y $(100 \leq X, Y, 10^{18})$ pelo menos metade dos dígitos iguais:

- 588 - interessante
- 123 - não interessante
- 1233 - interessante

DP de dígito

Quando precisamos operar sobre um intervalo muito grande de números e precisamos selecionar sobre características do número em si (divisibilidade, distribuição de dígitos, etc), pode ser útil fazer uma DP "montando" o número dígito a dígito - a DP de dígitos.

Tipicamente essas DPs tem um estado determinante - qual dígito do número você está montando no momento.

Problema Odometer - estados

Calcularemos $f(d, Z)$ - quantos números naturais menores que Z tem mais da metade dos dígitos iguais a d. A solução do problema será $f(d, Y) - f(d, X - 1)$ para todo d de 0 à 9.

Problema Odometer - estados

Calcularemos $f(d, Z)$ - quantos números naturais menores que Z tem mais da metade dos dígitos iguais a d. A solução do problema será $f(d, Y) - f(d, X - 1)$ para todo d de 0 à 9.

Os estados da DP são:

- `pos` - qual dígito estamos alterando (entre 0 e 18).
- `zero-esquerda` - já começamos a montar o número ou ainda estamos fazendo zeros à esquerda?
- `limite` - estamos colados no limite Z ou já estamos abaixo?
- `k` - quantos dígitos de d o número tem a mais que os outros dígitos?

Problema Odometer - limite

	limite	limite		
Z -	5	2	4	1
	5	2		

Ficamos abaixo do limite. $\longrightarrow$ 2 1

Podemos escolher qualquer dígito.

Problema Odometer - zero à esquerda

Z -	5	2	4	1
	0	0	0	1 ←
	zero esq	zero esq	zero esq	Começamos o número, podemos passar a contar os dígitos repetidos

Problema Odometer - k

d = 2

	k=0	k=0	k=1	k=0
Z -	5	2	4	1
	0	0		
			2	1

k não negativo significa que mais da metade dos dígitos são bons

Problema Odometer - acertando a contagem

Precisamos levar em conta os casos em que tem dois dígitos que aparecem metade das vezes cada, então temos que ter um estado a mais na DP para contar esse caso (que contamos repetido) e remover para acertar a contagem.

Código explicado USACO

Dicas de DP de dígito

- São muitos ifs, geralmente é melhor fazer top down.
- Usar inclusão-exclusão diminui muito a complexidade da solução.
- Cuidado com limites, um número até 10^x tem $x + 1$ dígitos! Pode compensar colocar dígitos a mais pra não ter que pensar nisso.
- Cuidado com zeros à esquerda.

Problema Counting Permutations

Dado n ($1 \leq n \leq 10^3$), conte (módulo $10^9 + 7$) o número de permutações de tamanho n tal que não haja elementos adjacentes com diferença 1.

- $(2, 4, 1, 3)$ - deve ser contada
- $(1, 2, 4, 3)$ - não deve ser contada

A ideia intuitiva (e ruim)

Podemos montar a sequência número a número $1, 2, \ldots, n$, escolhendo, em princípio, somente a posição relativa dos números.

- (1) - bom
- $(1, 2)$ - ruim
- $(1, 3, 2)$ - ruim
- $(1, 3, 2, 4)$ - ruim
- $(1, 3, 5, 2, 4)$ - bom

Não conseguimos guardar a posição real dos elementos e, somente colocarmos um a um, faz faltar informação.

DP de componentes conexos - a ideia

Ao invés de somente manter a ordem relativa, manteremos a ordem relativa dentro e fora dos componentes, mas nunca adicionaremos elementos no meio de componentes.

- (1) - bom
- $(1), (2)$ - bom (nenhuma regra é violada dentro dos componentes)
- $(1), (3), (2)$ - bom (nenhuma regra é violada dentro dos componentes)
- $(1), (3), (2, 4)$ - bom (nenhuma regra é violada dentro dos componentes)
- $(1), (3, 5, 2, 4)$ - bom
- $(1, 6, 3, 5, 2, 4)$ - bom

Ao invés de manter a invariante (não ter elementos consecutivos com diferença 1) sobre a sequência toda, garantimos que isso vale apenas para os "componentes", quando tivermos um único componente no fim, ele respeitará a propriedade que queremos!

DP de componentes conexos - transições

Teremos 3 tipos de transições válidas dentro dessa lógica:

- Criamos um novo componente conexo entre outros componentes (mantendo a ordem relativa):
  - $(1), (2) \rightarrow (1), (3), (2)$ (3 formas de fazer)
- Adicionamos um item ao começo ou fim de um componente conexo:
  - $(1), (3), (2) \rightarrow (1), (3), (2, 4)$ (6 formas de fazer - 4 válidas)
- Unimos dois componentes colocando um elemento no meio:
  - $(1), (3), (2, 4) \rightarrow (1), (3, 5, 2, 4)$ (2 formas de fazer - 2 válidas)

DP de componentes conexos - prova de corretude

Como garantimos que estamos olhando exatamente uma vez para cada permutação?

Para provar, faremos uma construção de uma permutação arbitrária p - mostraremos que, ao colocar elemento a elemento $(1, 2, \ldots, n)$, sempre temos exatamente uma forma de fazê-lo para chegar em p, logo consideramos cada permutação uma exatamente uma vez.

DP de componentes conexos - prova de corretude

Demonstração.

A prova segue por indução. Temos uma permutação p de n elementos, e vamos construí-la elemento a elemento. Considere que estamos no i-ésimo elemento $(1, \ldots, i, \ldots, n)$ e já temos uma sequência de "componentes conexos" - p' composta pelos elementos $1, \ldots, i-1$. Seguem os 3 casos possíveis para posicionar o elemento i:

- Em p, i fica entre dois elementos $a, b < i$.
  - $p = (\ldots, a, i, b, \ldots)$.
  - $p' = ((\ldots, a), i, (b, \ldots)) \to ((\ldots, a, i, b, \ldots))$.
  - i vai ter que ser colocado unindo os componentes de a e de b - como os dois já estão em p' já que $a, b < i$.

$\square$

DP de componentes conexos - prova de corretude

Demonstração.

- Em p, i fica entre dois elementos $a < i, B > i$.
  - $p = (\ldots, a, i, B, \ldots)$.
  - $p' = ((\ldots, a), i, (\ldots)) \to ((\ldots, a, i), (\ldots))$.
  - i vai ter que ser colocado no componente de a em p' já $a < i$. i não pode se juntar a outro componente pois ele precisa ser a borda de seu componente para poder se unir a B no futuro.

$\square$

DP de componentes conexos - prova de corretude

Demonstração.

- Em p, i fica entre dois elementos $A, B > i$.
  - $p = (\ldots, A, i, B, \ldots)$.
  - $p' = ((\ldots), i, (\ldots)) \to ((\ldots), (i), (\ldots))$.
  - i vai ter que ficar em um novo componente (na posição compatível na ordem relativa). i não pode se juntar a outro componente pois ele precisa estar na beira do seu componente nos dois lados a borda de seu componente para poder se unir a A e B no futuro.

$\square$

Problema Counting Permutations - solução

- A DP de componentes conexos é composta por dois estados - quantos elementos já colocamos e quantos componentes conexos temos.
- Precisaremos de dois estados extras - quantas bordas temos proibidas nos cantos e no meio da permutação (tem como fazer com um só mas eu achei assim mais intuitivo).

Problema Counting Permutations - transições

```
// cria novo componente quando não tem nada
if (comps == 0) {
    ret += dp(/*used=*/used + 1,
              /*comps=*/1,
              /*blocked_border=*/2,
              /*blocked_middle=*/0);
    ret %= MOD;
}
```

Problema Counting Permutations - transições

```
// cria um componente no canto
if (comps >= 1) {
    ret += 2 *
           dp(/*used=*/used + 1,
              /*comps=*/comps + 1,
              /*blocked_border=*/1,
              /*blocked_middle=*/1);
    ret %= MOD;
}
```

Problema Counting Permutations - transições

```
// cria um componente no meio
if (comps >= 2) {
    ret += (comps - 1) *
           dp(/*used=*/used + 1,
              /*comps=*/comps + 1,
              /*blocked_border=*/0,
              /*blocked_middle=*/2);
}
```

Problema Counting Permutations - transições

```
// une dois componentes
if (comps >= 2) {
    ret += (comps - 1 - blocked_middle) *
           dp(/*used=*/used + 1,
              /*comps=*/comps - 1,
              /*blocked_border=*/0,
              /*blocked_middle=*/0);
}
```

Problema Counting Permutations - transições

```
// adiciona a um componente (no canto)
if (comps >= 1) {
    ret += (2 - blocked_border) *
           dp(/*used=*/used + 1,
              /*comps=*/comps,
              /*blocked_border=*/1,
              /*blocked_middle=*/0);
    ret %= MOD;
}
```

Problema Counting Permutations - transições

```
// adiciona a um componente (no meio)
if (comps >= 2) {
    ret += (2*comps - 2 - blocked_middle) *
           dp(/*used=*/used + 1,
              /*comps=*/comps,
              /*blocked_border=*/0,
              /*blocked_middle=*/1);
    ret %= MOD;
}
```

Problema Counting Permutations - caso base

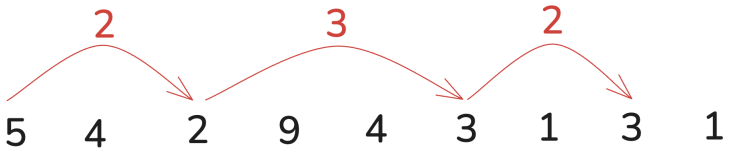
```
if (used == n) {
    return comps == 1;
}
```

A resposta está em $dp(0, 0, 0, 0)$.

Problema Treasure Hunter

Temos um vetor de valores $c = (c_1, \ldots, c_n)$ $1 \leq n \leq 3 * 10^4$.

Começamos fazendo um pulo para a posição d e seguimos para a direita (sem destino). O primeiro pulo tem comprimento d, depois de um pulo com comprimento x, o próximo pulo deve ter comprimento $x - 1$ ou $x + 1$ (os pulos devem ser positivos).



Maximize a soma dos elementos atingidos.

Problema Treasure Hunter

A DP intuitiva é $f(i, j)$ é o máximo de valor que conseguimos atingir dado que estamos na posição i e o último pulo foi de comprimento j.

Problema Treasure Hunter

A DP intuitiva é $f(i, j)$ é o máximo de valor que conseguimos atingir dado que estamos na posição i e o último pulo foi de comprimento j.

Contudo, nesse problema ao fazer a conta vemos que faríamos na casa de 10^9 operações $(1 \leq i, j \leq d, n \leq 3 * 10^4)$.

Problema Treasure Hunter

Contudo, seja l o menor pulo que você deu e r o maior pulo dado:

Os pulos distintos são $l, l+1, l+2, \ldots, r \leq 3 * 10^4$.

Fazendo a soma:
$l + (l+1) + (l+2) + \ldots + (l+245) \geq 1 + 2 + \ldots 245 = \frac{245*246}{2} = 30135 > 3 * 10^4$.

Logo conseguimos atingir no máximo 245 valores de j - nossa DP é viável.

Não alcançamos todos os estados

- Sempre pense sobre quais estados são atingíveis.
- Dependendo do caso (quando sabemos que são poucos estados atingíveis, mas não sabemos exatamente quais), podemos fazer uma DP "esparsa", guardando resultados em um map.

Lidando com ciclos

E quando a DP tem ciclos?

- Criar grafo de SCC para transformar o grafo em uma DAG.
- Usar a relação de recorrência para acabar com a relação cíclica.
- Linearização.
- Chuta um valor e *mágica numérica*/Eliminação Gaussiana.

Lidando com ciclos

E quando a DP tem ciclos?

- Criar grafo de SCC para transformar o grafo em uma DAG.
- Usar a relação de recorrência para acabar com a relação cíclica.
- Linearização.
- Chuta um valor e *mágica numérica*/Eliminação Gaussiana. Assunto para outra aula

Lidando com ciclos

E quando a DP tem ciclos?

- Criar grafo de SCC para transformar o grafo em uma DAG.
- Usar a relação de recorrência para acabar com a relação cíclica.
- Linearização.
- Chuta um valor e *mágica numérica*/Eliminação Gaussiana. Assunto para outra aula
- Talvez não dê pra resolver com DP!

Problema Sticker album - relação de recorrência

Queremos preencher um álbum com N figurinhas (todas são iguais).

Cada pacote tem de A até B figurinhas ($0 \leq A, B \leq 10^6$) distribuídas com probabilidade uniforme. Qual é o número esperado de pacotes que devemos abrir?

Problema Sticker album - relação de recorrência

Queremos preencher um álbum com N figurinhas (todas são iguais).

Cada pacote tem de A até B figurinhas ($0 \leq A, B \leq 10^6$) distribuídas com probabilidade uniforme. Qual é o número esperado de pacotes que devemos abrir?

Seja E_x o número esperado de pacotes que precisamos abrir para conseguir x figurinhas.

Seja p a probabilidade de conseguir y cartinhas $A \leq y \leq B$. $p = \frac{1}{B-A+1}$.

$$E_x = \begin{cases} 0 & x = 0 \\ 1 + \sum_{y \in [A,B]} p * E_{\max(0,x-y)} & x > 0 \end{cases}$$

Problema Sticker album - relação de recorrência

Queremos preencher um álbum com N figurinhas (todas são iguais).

Cada pacote tem de A até B figurinhas ($0 \leq A, B \leq 10^6$) distribuídas com probabilidade uniforme. Qual é o número esperado de pacotes que devemos abrir?

Seja E_x o número esperado de pacotes que precisamos abrir para conseguir x figurinhas.

Seja p a probabilidade de conseguir y cartinhas $A \leq y \leq B$. $p = \frac{1}{B-A+1}$.

$$E_x = \begin{cases} 0 & x = 0 \\ 1 + \sum_{y \in [A,B]} p * E_{\max(0,x-y)} & x > 0 \end{cases}$$

E se A for 0?

Problema Sticker album - relação de recorrência

Quando $A = 0$ e $x > 0$:

$$E_x = 1 + p \cdot E_x + \sum_{y \in [A+1,B]} p \cdot E_{\max(0,x-y)}$$

$$E_x - p \cdot E_x = 1 + \sum_{y \in [A+1,B]} p \cdot E_{\max(0,x-y)}$$

$$(1-p) \cdot E_x = 1 + \sum_{y \in [A+1,B]} p \cdot E_{\max(0,x-y)}$$

$$E_x = \frac{1 + \sum_{y \in [A+1,B]} p \cdot E_{\max(0,x-y)}}{1-p}$$

Linearização - House Robber II

Temos N casas em um ciclo, cada casa tem um valor a ser roubado.
Você quer saber qual é o máximo valor que você consegue roubar sem roubar duas casas consecutivas, já que isso dispararia o alarme.

- Se fosse em uma linha, basta fazer uma DP(pos, pegou antes ou não).

Linearização - House Robber II

Temos N casas em um ciclo, cada casa tem um valor a ser roubado.
Você quer saber qual é o máximo valor que você consegue roubar sem roubar duas casas consecutivas, já que isso dispararia o alarme.

- Se fosse em uma linha, basta fazer uma DP(pos, pegou antes ou não).
- Podemos fazer o mesmo com o ciclo, mas rodamos a DP duas vezes, em uma delas pegamos 1 e esquecemos o elemento N e na outra não pegamos 1 e permitimos pegar o elemento N

Formulário feedback

https://forms.gle/oVMoUkJvbJVoZuYh8

Referências

- Digit DP USACO
- Blog Connected component DP por humbertoyusta
- Blog Non trivial DP tecnniques por zscoder
- DP on broken profile USACO